



Universidad Autónoma de Baja California
Facultad de Ingeniería, Arquitectura y Diseño
Ingeniería en Software y Tecnologías Emergentes

Aplicaciones Web

Grupo 961

Paulina Mendoza Quiroz

372532

Jennifer Salgado Pacheco

375100

Gerardo Yepez Tsuchiya

346176

Proyecto

Ensenada, B.C. 02 de Marzo 2026

Índice

1. Introducción

1.1. Propósito del Proyecto

1.2. Alcance del Proyecto

2. Análisis de Requisitos

2.1. Requisitos Funcionales

2.2 Requisitos No Funcionales

2.3. Requisitos de usabilidad

3. Planificación del Proyecto

3.1. Cronograma

3.2. Entregables

4. Recursos

4.1. Tecnológicos

Introducción

El presente documento describe el desarrollo de una aplicación web de gestión de tareas (Task Manager), construida como proyecto de la materia Aplicaciones Web en la UABC. El sistema tiene como objetivo principal ofrecer a los usuarios una plataforma centralizada donde puedan organizar, asignar y dar seguimiento a tareas dentro de proyectos colaborativos.

El backend de la aplicación es desarrollado con NestJS, exponiendo una API REST que será consumida por un frontend construido con Vite + React. La base de datos es gestionada mediante Prisma ORM sobre PostgreSQL, alojado en Supabase.

1.2 Alcance del Proyecto

El sistema contempla los siguientes módulos principales:

- Autenticación de usuarios (registro, inicio de sesión, tokens JWT)
- Gestión de proyectos (crear, editar, eliminar, listar proyectos propios)
- Gestión de tareas (crear, asignar, cambiar estado, eliminar tareas por proyecto)
- Gestión de usuarios (consultar usuarios para asignación de tareas)

Quedan fuera del alcance para la entrega de mayo las siguientes funcionalidades:

- Notificaciones en tiempo real
- Integración con servicios externos de calendario
- Módulo de reportes o analíticas avanzadas
- Aplicación móvil nativa

2. Análisis de Requisitos

2.1 Requisitos Funcionales

Los siguientes requisitos funcionales definen el comportamiento esperado del sistema:

ID	Módulo	Descripción
RF-01	Auth	El sistema debe permitir el registro de nuevos usuarios con nombre, correo y contraseña.
RF-02	Auth	El sistema debe autenticar usuarios mediante correo y contraseña, retornando un token JWT.
RF-03	Auth	Las rutas protegidas deben validar el token JWT antes de procesar la solicitud.
RF-04	Proyectos	El usuario autenticado puede crear, editar, consultar y eliminar sus proyectos.
RF-05	Tareas	El usuario puede crear tareas dentro de un proyecto, asignándoles título, descripción, prioridad y fecha límite.
RF-06	Tareas	El usuario puede cambiar el estado de una tarea (pendiente, en progreso, completada).
RF-07	Tareas	El usuario puede asignar tareas a otros usuarios registrados en el sistema.
RF-08	Usuarios	El sistema debe exponer un endpoint para listar usuarios disponibles para asignación.

2.2 Requisitos No Funcionales

Los siguientes requisitos definen atributos de calidad del sistema:

ID	Categoría	Descripción
RNF01	Seguridad	Las contraseñas deben almacenarse hasheadas con bcrypt, nunca en texto plano.
RNF02	Seguridad	La comunicación entre frontend y backend debe estar protegida mediante HTTPS en producción.
RNF06	Mantenibilidad	El código debe seguir la estructura de módulos de NestJS: Module / Controller / Service.
RNF03	Rendimiento	La API debe responder en menos de 500ms bajo carga normal de uso académico.
RNF04	Disponibilidad	La base de datos estará alojada en Supabase con disponibilidad del servicio gratuito (~99%).
RNF05	Escalabilidad	La arquitectura modular de NestJS permite agregar nuevas funcionalidades sin refactorizar el núcleo.

2.3 Requisitos de Usabilidad

Desde la perspectiva del backend, los requisitos de usabilidad impactan principalmente en el diseño de la API:

- Los endpoints deben retornar mensajes de error claros y códigos HTTP apropiados (400, 401, 403, 404, 500).
- Las respuestas deben tener una estructura consistente para facilitar el consumo desde el frontend.
- La API debe habilitar CORS para permitir peticiones desde el servidor de desarrollo de Vite (puerto 5173).
- La documentación de endpoints debe ser accesible (se considerará Swagger/OpenAPI como entregable opcional).

3. Planificación del Proyecto

3.1 Cronograma

El proyecto tiene como fecha límite el mes de mayo de 2026. A continuación se presenta el cronograma planificado por fases:

Entregable	Descripción
API REST funcional	Servidor NestJS con todos los módulos implementados y corriendo en producción o local.
Base de datos	Esquema Prisma con modelos User, Project y Task correctamente relacionados en Supabase.
Autenticación JWT	Sistema de registro, login y protección de rutas mediante tokens.
Código fuente	Repositorio con código limpio, estructurado por módulos y con variables de entorno documentadas.
Documentación	Este documento más un README con instrucciones de instalación y ejecución.

4. Recursos

4.1 Recursos Tecnológicos

A continuación se listan las tecnologías, herramientas y servicios utilizados en el proyecto:

Categoría	Tecnología / Herramienta	Versión	Rol en el Proyecto
Runtime	Node.js	LTS (22.x)	Entorno de ejecución del servidor backend.
Framework	NestJS	10.x	Framework principal para la API REST.
Lenguaje	TypeScript	5.x	Lenguaje de programación del backend.
ORM	Prisma	7.x	Comunicación y modelado de la base de datos.
Base de datos	PostgreSQL (Supabase)	Free tier	Almacenamiento persistente de datos.
Autenticación	@nestjs/jwt + Passport	Latest	Generación y validación de tokens JWT.
Seguridad	bcrypt	Latest	Hashing de contraseñas de usuarios.
Validación	class-validator + class-transformer	Latest	Validación de DTOs en los endpoints.
Frontend	Vite + React	5.x / 18.x	Interfaz de usuario (a cargo del equipo de frontend).
Control de versiones	Git / GitHub	-	Versionado y colaboración del código.
Editor	Visual Studio Code	-	Entorno de desarrollo.